HTTP/HTTPS Protocols

HTTP (Hypertext Transfer Protocol)

HTTP is the foundation of data communication on the World Wide Web. It's an application-layer protocol that defines how messages are formatted and transmitted between web browsers (clients) and web servers.

Key characteristics:

- **Request-Response Model**: The client sends an HTTP request, and the server responds with the requested resource (like an HTML page, image, or data)
- **Stateless**: Each request is independent; the server doesn't retain information about previous requests by default (though cookies and sessions work around this)
- **Text-Based**: HTTP messages are human-readable text
- **Port 80**: HTTP typically uses TCP port 80

HTTP Request Structure

- Method (GET, POST, PUT, DELETE)
- URL
- Headers
- Body (optional)

HTTP Response Structure

- Status Code (200, 404, 500)
- Headers
- Body

Common HTTP Methods:

- GET: Retrieve data from a server
- POST: Submit data to be processed
- PUT: Update existing resources
- DELETE: Remove resources
- HEAD: Get headers without the body
- PATCH: Partially modify resources

HTTP Status Codes:

- 2xx (Success): 200 OK, 201 Created
- 3xx (Redirection): 301 Moved Permanently, 304 Not Modified
- 4xx (Client Error): 404 Not Found, 403 Forbidden
- 5xx (Server Error): 500 Internal Server Error, 503 Service Unavailable

How do we manage login sessions?

- Cookies
- JWT tokens
- Session storage (Redis, DB)

Why is HTTP stateless?

HTTP is stateless by design to keep the protocol simple and to enable scalability. Each request contains all the information required to process it, allowing servers to handle requests independently. This makes horizontal scaling, load balancing, and fault tolerance much easier.

HTTPS (HTTP Secure)

HTTPS is HTTP with encryption. It uses TLS (Transport Layer Security), formerly SSL, to encrypt the communication between client and server.

Key differences from HTTP:

- **Encryption**: All data transmitted is encrypted, protecting it from eavesdropping
- **Authentication**: Verifies you're connecting to the actual website (not an imposter)
- **Data Integrity**: Ensures data hasn't been tampered with during transmission
- **Port 443**: HTTPS typically uses TCP port 443
- **Certificate Required**: Servers need an SSL/TLS certificate from a trusted Certificate Authority

How HTTPS Works:

1. Client initiates connection to server
2. Server sends its SSL/TLS certificate
3. Client verifies the certificate with a Certificate Authority
4. Client and server negotiate encryption methods
5. They establish an encrypted connection (TLS handshake)
6. Encrypted data transmission begins

Why HTTPS Matters:

- Protects sensitive information (passwords, credit cards, personal data)
- Prevents man-in-the-middle attacks
- Builds user trust (browsers mark HTTP sites as "Not Secure")
- SEO benefits: Search engines favor HTTPS sites
- Required for modern web features (geolocation, service workers, etc.)

TLS Handshake (Very Important for Interviews)

Before sending real data:

1. Client says hello
2. Server sends certificate
3. Client verifies certificate
4. Key exchange happens
5. Encrypted communication starts

This handshake adds:

- Extra latency (important in system design!)
- CPU overhead

That’s why:

- We use CDN
- We use TLS termination at load balancer

What Is Idempotency?

A request is **idempotent** if: Performing it multiple times has the same effect as performing it once.

Important:

- The **response may differ**
- But the **final system state remains the same**

Simple Example

✅ **Idempotent Operation** DELETE /users/123

Call it once → user deleted
Call it 5 times → user still deleted

Final state = same

❌ **Non-Idempotent Operation** POST /orders

Call once → 1 order created
Call twice → 2 orders created

Final state changed.

🔹 HTTP Methods and Idempotency

Method	Idempotent?	Why
GET	✅ Yes	Fetching doesn't change state
PUT	✅ Yes	Replaces resource completely
DELETE	✅ Yes	Deleting multiple times same result
POST	❌ No	Creates new resource each time
PATCH	⚠️ Usually No	Partial update may accumulate changes

🔥 Why Idempotency Matters in Distributed Systems

This is where it becomes critical.

Imagine:

Client → Server
Network fails
Client retries

If operation is not idempotent:

You might:

- Charge a credit card twice
- Create duplicate orders
- Send duplicate emails

That's bad.

How We Solve This?

We use something called an **Idempotency Key**.

Client sends:

```
bash

POST /payments
Idempotency-Key: 12345
```

Server:

- Stores result of first request
- If same key comes again → returns previous response
- Does NOT reprocess

This makes POST behave idempotently.